LAB 3 · LECTURE 3

Lecture 3 Lab: Build a Search Engine with NumPy

Vector Databases Course | Estimated time: 60-75 minutes | Platform: Google Colab (free)

Goal

You will implement cosine, dot product and Euclidean distance by hand, build a complete semantic search engine in NumPy with no database, and measure how exact search behaves as the data grows.

- Implement the three metrics and verify the lecture's worked example.
- Build a reusable `SearchEngine` class with fast top-k selection.
- Prove that normalization makes cosine, dot product and L2 rank identically.
- Evaluate each metric with precision@3, with and without normalization.
- Benchmark exact search against N and reproduce the curse of dimensionality.

Setup

Create a new Colab notebook and run each block in its own cell, in order.

Cell 1: install and import

```
!pip install -q sentence-transformers numpy pandas matplotlib

import time
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sentence_transformers import SentenceTransformer

rng = np.random.default_rng(0)
```

Part 1: The metrics by hand

Use plain NumPy operations only. These are the vectors from the lecture: a = (1, 3), b = (3, 1), c = (2, 6) = 2a.

Cell 2: implement and check

```
def dot(a, b):    return float(np.sum(a * b))
def norm(a):      return float(np.sqrt(np.sum(a * a)))
def cosine(a, b): return dot(a, b) / (norm(a) * norm(b))
def l2(a, b):     return norm(a - b)

a, b, c = np.array([1, 3.]), np.array([3, 1.]), np.array([2, 6.])
for name, x, y in [("a,b", a, b), ("a,c", a, c)]:
    print(f"{name}: cos={cosine(x, y):.2f} dot={dot(x, y):.0f} l2={l2(x, y):.2f}")
```

Expected: a,b gives cosine 0.60, dot 6, L2 2.83. a,c gives cosine 1.00, dot 20, L2 3.16.

YOUR TURN Before running, predict how b and c rank for the query a under each metric. Then check. Which metric disagrees with the other two, and why?

Part 2: A search engine

Thirty help-center titles across five topics. Some sentences overlap on purpose (refunds appear under billing and returns).

Cell 3: corpus

```
data = [
 ("account", "How to reset your account password"),
 ("account", "Change the email address on my profile"),
 ("account", "Delete my account permanently"),
 ("account", "Enable two-factor authentication"),
 ("account", "I forgot my login credentials"),
 ("account", "Update my display name"),
 ("billing", "Why was I charged twice this month"),
 ("billing", "Download my invoice as a PDF"),
 ("billing", "Update my credit card details"),
 ("billing", "How to cancel my subscription"),
 ("billing", "Refund policy for cancelled orders"),
 ("billing", "Apply a discount code at checkout"),
 ("shipping", "Where is my package right now"),
 ("shipping", "How long does international delivery take"),
 ("shipping", "Change the delivery address after ordering"),
 ("shipping", "Shipping costs for large items"),
 ("shipping", "My parcel arrived damaged"),
 ("shipping", "Track an order without an account"),
 ("technical", "The app crashes when I open settings"),
 ("technical", "Wi-Fi keeps disconnecting every few minutes"),
 ("technical", "Laptop battery drains too quickly"),
 ("technical", "My screen flickers after the update"),
 ("technical", "How to back up my phone"),
 ("technical", "Bluetooth headphones will not pair"),
 ("returns", "Start a return for a defective item"),
 ("returns", "How to exchange a product for a different size"),
 ("returns", "Is a return label free"),
 ("returns", "Return window for electronics"),
 ("returns", "Where to drop off my return"),
 ("returns", "Refund timeline after a return"),
]
topics = [t for t, _ in data]; texts = [s for _, s in data]
model = SentenceTransformer("all-MiniLM-L6-v2")
D = model.encode(texts, normalize_embeddings=True)
print(D.shape)
```

Cell 4: the SearchEngine class

```
class SearchEngine:
    def __init__(self, texts, topics, model, D):
        self.texts, self.topics, self.model, self.D = texts, topics, model, D

    def score(self, q, metric):
        if metric == "dot":
            return self.D @ q
        if metric == "cosine":
            norms = np.linalg.norm(self.D, axis=1) * np.linalg.norm(q)
            return (self.D @ q) / norms
        if metric == "l2":
            return -np.linalg.norm(self.D - q, axis=1)     # negated: higher = closer
        raise ValueError(metric)

    def search(self, query, k=5, metric="cosine"):
        q = self.model.encode(query, normalize_embeddings=True)
        s = self.score(q, metric)
        top = np.argpartition(-s, k)[:k]
        top = top[np.argsort(-s[top])]
        return pd.DataFrame({"rank": range(1, k + 1), "score": s[top].round(3),
                             "text": [self.texts[i] for i in top],
                             "topic": [self.topics[i] for i in top]})

engine = SearchEngine(texts, topics, model, D)
engine.search("I can't sign in to my profile", k=5)
```

YOUR TURN Run five queries of your own, including one with no word overlap with any title. Does the engine behave as the lecture predicts?

Part 3: Normalization makes the metrics agree

Cell 5: algebra and ranking check

```
u = rng.normal(size=(2, 384)); u /= np.linalg.norm(u, axis=1, keepdims=True)
lhs = np.sum((u[0] - u[1]) ** 2); rhs = 2 - 2 * np.dot(u[0], u[1])
print("|a-b|^2:", round(lhs, 4), "2-2a.b:", round(rhs, 4), np.isclose(lhs, rhs))

q = "when will my order get here"
orders = {}
for m in ["cosine", "dot", "l2"]:
    orders[m] = engine.search(q, k=5, metric=m)["text"].tolist()
print("same ranking for all three:", orders["cosine"] == orders["dot"] == orders["l2"])
```

Part 4: Evaluate with precision@3

Each query has an expected topic. Precision@3 is the fraction of the top three results with that topic. Some models return vectors of varying length, so we simulate that by scaling each document vector by a random factor.

Cell 6: evaluation with and without normalization

```
eval_set = [
    ("I can't sign in to my profile", "account"),
    ("switch on extra security for logging in", "account"),
    ("Why did the payment go through two times", "billing"),
    ("get a copy of my receipt", "billing"),
    ("when will my order get here", "shipping"),
    ("the box was broken when it arrived", "shipping"),
    ("my phone keeps losing the internet", "technical"),
    ("my computer's charge does not last long", "technical"),
    ("send back something I bought", "returns"),
]

def precision_at_3(eng, metric):
    hits = []
    for q, t in eval_set:
        res = eng.search(q, k=3, metric=metric)
        hits.append((res["topic"] == t).mean())
    return round(float(np.mean(hits)), 3)

scaled = D * rng.uniform(0.5, 3.0, size=(len(D), 1))   # unnormalized
rows = []
for label, mat in [("normalized", D), ("random scale", scaled)]:
    eng = SearchEngine(texts, topics, model, mat)
    row = {"vectors": label}
    for m in ["cosine", "dot", "l2"]:
        row[m] = precision_at_3(eng, m)
    rows.append(row)
pd.DataFrame(rows)
```

YOUR TURN Complete the table from your output, then explain why dot and L2 changed on scaled vectors while cosine did not.

Vectors	Cosine	Dot	L2
normalized			
random scale			

Part 5: Fast top-k and batching

Cell 7: argsort vs argpartition

```
scores = rng.normal(size=1_000_000).astype("float32"); k = 10
t0 = time.perf_counter()
full = np.argsort(-scores)[:k]
t_sort = time.perf_counter() - t0
t0 = time.perf_counter()
part = np.argpartition(-scores, k)[:k]; part = part[np.argsort(-scores[part])]
t_part = time.perf_counter() - t0
print("same result:", np.array_equal(full, part))
print(f"argsort {t_sort*1000:.0f} ms | argpartition {t_part*1000:.0f} ms")
```

Cell 8: one query at a time vs a batch

```
Dn = rng.normal(size=(200_000, 384)).astype("float32")
Dn /= np.linalg.norm(Dn, axis=1, keepdims=True)
Q = rng.normal(size=(100, 384)).astype("float32")
Q /= np.linalg.norm(Q, axis=1, keepdims=True)

t0 = time.perf_counter()
for q in Q: _ = Dn @ q
t_loop = time.perf_counter() - t0
t0 = time.perf_counter(); S_all = Q @ Dn.T; t_batch = time.perf_counter() - t0
print(f"loop {t_loop:.2f} s | batch {t_batch:.2f} s | scores shape {S_all.shape}")
```

YOUR TURN Which is faster, and why? What limits how many queries you can batch at once?

Part 6: Exact search does not scale

Cell 9: time per query vs N

```
sizes = [10_000, 50_000, 200_000, 500_000]
q = Q[0]; times = []
for n in sizes:
    X = rng.normal(size=(n, 384)).astype("float32")
    t0 = time.perf_counter()
    for _ in range(20): _ = X @ q
    times.append((time.perf_counter() - t0) / 20 * 1000)
    del X

plt.loglog(sizes, times, marker="o")
plt.xlabel("N (vectors)"); plt.ylabel("ms per query")
plt.grid(alpha=0.3); plt.title("Exact search time vs N"); plt.show()
print([f"{n:,}: {t:.1f} ms" for n, t in zip(sizes, times)])
```

YOUR TURN Fit the growth: when N is multiplied by 10, by how much does the time grow? Extrapolate to 100 million vectors. Is that acceptable for a web search box?

Part 7: The curse of dimensionality

Cell 10: farthest over nearest distance

```
for d in [2, 10, 100, 1000]:
    X = rng.normal(size=(500, d)); q = rng.normal(size=d)
    dist = np.linalg.norm(X - q, axis=1)
    print(f"d={d:5d}  farthest/nearest = {dist.max() / dist.min():.2f}")
```

YOUR TURN Predict the trend before running. What does it imply for score thresholds on 384-dimensional embeddings?

Part 8: Break it

Cell 11: a document that cheats with its length

```
boost = D.copy()
i = texts.index("Delete my account permanently")
boost[i] *= 10                                    # unusually long vector
hack = SearchEngine(texts, topics, model, boost)

q2 = "when will my order get here"
for m in ["cosine", "dot", "l2"]:
    print(m, hack.search(q2, k=3, metric=m)["text"].tolist())
```

YOUR TURN Which metric lets the cheating document in, and which ones resist? Which would you pick for a public search box where documents are uploaded by users?

Reflection questions

- Why did argpartition beat argsort, and why do we still sort the k survivors?
- Why is precision@3 on 9 queries not enough to claim one metric is better? What would you do instead?
- Chroma's default distance is squared L2. When would that be a problem for your data, and how would you check?
- Your engine takes 80 ms per query at 1 million vectors. List two ways to reduce that without changing the answer's quality, then two that trade quality for speed. (Lecture 4 returns to this.)

Deliverable

Submit the executed notebook, the benchmark plot, the completed table, the answers to each YOUR TURN box, and short answers to the reflection questions.

Troubleshooting

- **Out of memory in Cell 9:** remove 500,000 from sizes, or restart the runtime and run fewer cells.
- **argpartition ValueError:** k must be smaller than the number of documents.
- **Timings or precision differ from classmates:** hardware and model versions vary; compare patterns, not absolute numbers.